ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI
XÂY DỰNG HỆ THỐNG CHATBOT TELEGRAM GỢI Ý ĐỊA ĐIỂM ĐI CHƠI SỬ DỤNG N8N & DOCKER

Giảng viên hướng dẫn:	TS Đỗ Như Tài
Sinh viên :	Mssv:
Nguyễn Việt Đức	3122411044
Nguyễn Huỳnh Phương Lộc	3122411117
Nguyễn Phạm Hoàng Quân	3122411169
Nguyễn Trần Quốc Tiến	3122411208

Thành phố Hồ Chí Minh, tháng 2 năm 2026

LỜI MỞ ĐẦU

Trong cuộc sống hiện đại, nhu cầu giải trí và thư giãn không chỉ diễn ra vào cuối tuần mà có thể phát sinh vào bất kỳ ngày nào trong tuần. Sau giờ học hoặc làm việc, nhiều người có nhu cầu tìm kiếm địa điểm ăn uống, cà phê, xem phim hoặc vui chơi để giảm căng thẳng. Tuy nhiên, việc lựa chọn địa điểm phù hợp thường mất nhiều thời gian do phải tìm kiếm thông tin từ nhiều nguồn khác nhau.

Với sự phát triển của các nền tảng nhắn tin như Telegram, người dùng có thể tương tác nhanh chóng thông qua chatbot để nhận thông tin theo thời gian thực. Bên cạnh đó, các công cụ tự động hóa như n8n cho phép xây dựng hệ thống xử lý dữ liệu linh hoạt mà không cần lập trình quá phức tạp. Kết hợp với Docker trong triển khai, hệ thống có thể hoạt động ổn định và dễ dàng mở rộng.

Xuất phát từ nhu cầu thực tế đó, nhóm thực hiện đề tài "Xây dựng chatbot Telegram gợi ý địa điểm vui chơi sử dụng n8n và Docker". Đề tài hướng đến việc xây dựng một hệ thống tự động có khả năng tiếp nhận yêu cầu từ người dùng, xử lý dữ liệu về vị trí và thời tiết, từ đó gợi ý địa điểm phù hợp vào bất kỳ thời điểm nào trong tuần.

PHÂN CÔNG CÔNG VIỆC

LỜI MỞ ĐẦU **2**
PHÂN CÔNG CÔNG VIỆC **3**
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 5
1.1. Giới thiệu bài toán 5
1.2. Mục tiêu của đề tài 5
1.3. Đối tượng và phạm vi nghiên cứu 5
1.4. Các chức năng chính của hệ thống 6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 6
2.1. Tổng quan về Chatbot Telegram 6
2.2. Giới thiệu công cụ n8n 7
2.3. Tổng quan về Docker 8
2.4. Các API sử dụng trong hệ thống 8
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 9
3.1. Kiến trúc tổng thể hệ thống 9
3.2. Luồng hoạt động của hệ thống 10
3.3. Thiết kế triển khai hệ thống trên Docker 12
3.4. Đánh giá thiết kế hệ thống 13
3.5 Sequence Diagram 14
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 14
4.1. Môi trường triển khai 14
4.2. Triển khai hệ thống bằng Docker Compose 15
4.3. Cấu hình Telegram Bot 15
4.4. Cấu hình Webhook thông qua Cloudflared 16
4.5. Kiểm thử hệ thống 16
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ 17
5.1. Kết quả triển khai và hoạt động của hệ thống 17
5.2 Kiểm thử chức năng yêu cầu của người dùng 20
5.2.1 Hỏi về vị trí hiện tại 20
5.2.2 Chatbot phản hồi về yêu cầu của người dùng 21
5.3. Kiểm thử hiệu năng và độ phản hồi 22
5.4. Đánh giá tổng quan hệ thống 23
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 24
6.1. Kết luận 24
6.2. Hạn chế của hệ thống 25
6.3. Hướng phát triển trong tương lai 25

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu bài toán

Hiện nay, khi có nhu cầu đi chơi, ăn uống hoặc giải trí, người dùng thường phải tìm kiếm thông tin trên các ứng dụng bản đồ hoặc mạng xã hội. Quá trình này có thể gây mất thời gian do phải lọc qua nhiều thông tin không cần thiết. Ngoài ra, yếu tố thời tiết hoặc khoảng cách địa lý cũng ảnh hưởng lớn đến việc lựa chọn địa điểm.

Do đó, bài toán đặt ra là xây dựng một hệ thống có khả năng tự động gợi ý địa điểm vui chơi phù hợp dựa trên vị trí hiện tại của người dùng và các điều kiện liên quan. Hệ thống cần hoạt động nhanh, dễ sử dụng và có thể truy cập thông qua nền tảng quen thuộc như Telegram.

1.2. Mục tiêu của đề tài

Đề tài được thực hiện với các mục tiêu sau:

- Xây dựng chatbot Telegram có khả năng tiếp nhận yêu cầu gợi ý địa điểm
- Tích hợp API thời tiết và API địa điểm để xử lý dữ liệu
- Tự động hóa luồng xử lý bằng công cụ n8n
- Triển khai hệ thống bằng Docker để đảm bảo tính linh hoạt và dễ quản lý

Ngoài ra, đề tài còn giúp sinh viên vận dụng kiến thức về container hóa, workflow automation và tích hợp API trong thực tế.

1.3. Đối tượng và phạm vi nghiên cứu

- ❖ Đối tượng sử dụng: cá nhân có nhu cầu tìm kiếm địa điểm vui chơi, ăn uống, giải trí trong tuần
- ❖ Phạm vi hệ thống:
    - ➢ Gợi ý các địa điểm như quán ăn, cà phê, rạp chiếu phim

- ➢ Tìm kiếm trong phạm vi bán kính nhất định quanh vị trí người dùng
- ➢ Phản hồi thông tin thông qua Telegram

Hệ thống chưa tập trung vào cá nhân hóa sâu theo hồ sơ người dùng mà chủ yếu dựa vào vị trí và điều kiện hiện tại

1.4. Các chức năng chính của hệ thống

Hệ thống chatbot Telegram được xây dựng với các chức năng chính như sau:

- Nhận yêu cầu gợi ý địa điểm từ người dùng
- Nhận và xử lý vị trí hiện tại
- Lấy thông tin thời tiết từ API bên ngoài
- Gợi ý danh sách địa điểm phù hợp với điều kiện thực tế
- Trả kết quả cho người dùng kèm theo thông tin chi tiết và liên kết bản đồ

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Tổng quan về Chatbot Telegram

Chatbot là một chương trình máy tính có khả năng giao tiếp với người dùng thông qua tin nhắn văn bản hoặc giọng nói. Trong đề tài này, chatbot được xây dựng trên nền tảng Telegram nhằm tạo ra một kênh tương tác nhanh chóng và tiện lợi.

Telegram Bot hoạt động thông qua Telegram Bot API. Khi người dùng gửi tin nhắn đến bot, hệ thống sẽ nhận dữ liệu thông qua webhook hoặc polling. Sau đó, dữ liệu được xử lý và phản hồi lại người dùng.

Ưu điểm của Telegram trong xây dựng chatbot:

- Miễn phí và dễ sử dụng
- Hỗ trợ gửi vị trí (location) trực tiếp
- Có tài liệu API rõ ràng
- Phù hợp cho việc thử nghiệm và triển khai cá nhân

Việc lựa chọn Telegram giúp hệ thống có thể tiếp cận người dùng một cách đơn giản mà không cần xây dựng giao diện web phức tạp.

2.2. Giới thiệu công cụ n8n

n8n là một công cụ workflow automation mã nguồn mở, cho phép kết nối và tự động hóa các dịch vụ khác nhau thông qua các node xử lý. Người dùng có thể thiết kế luồng xử lý dữ liệu bằng giao diện kéo thả mà không cần viết quá nhiều mã lập trình.

Trong hệ thống này, n8n đóng vai trò trung tâm xử lý dữ liệu, bao gồm:

- Nhận yêu cầu từ Telegram
- Gọi API thời tiết
- Gọi API địa điểm
- Xử lý và lọc dữ liệu
- Trả kết quả về cho người dùng

Ưu điểm của n8n:

- Mã nguồn mở, có thể tự host
- Dễ tích hợp nhiều API khác nhau
- Giao diện trực quan
- Linh hoạt trong việc xử lý logic

Việc sử dụng n8n giúp giảm đáng kể độ phức tạp khi xây dựng hệ thống tích hợp nhiều API.

2.3. Tổng quan về Docker

Docker là nền tảng cho phép đóng gói ứng dụng và các thành phần phụ thuộc vào trong container. Container giúp ứng dụng có thể chạy ổn định trên nhiều môi trường khác nhau mà không bị ảnh hưởng bởi sự khác biệt về hệ điều hành hoặc cấu hình.

Trong đề tài này, Docker được sử dụng để:

- Triển khai n8n dưới dạng container
- Quản lý các dịch vụ liên quan
- Đảm bảo hệ thống có thể chạy ổn định và dễ dàng di chuyển sang môi trường khác

Việc sử dụng Docker giúp quá trình triển khai trở nên đơn giản hơn, chỉ cần một số lệnh cơ bản để khởi động toàn bộ hệ thống.

2.4. Các API sử dụng trong hệ thống

Hệ thống sử dụng các API bên ngoài để cung cấp dữ liệu cần thiết:

Weather API

Cung cấp thông tin thời tiết hiện tại tại vị trí người dùng, bao gồm:

- Nhiệt độ
- Trạng thái thời tiết (nắng, mưa, nhiều mây...)
- Độ ẩm

Thông tin này được sử dụng để điều chỉnh loại địa điểm gợi ý (ví dụ: nếu trời mưa thì ưu tiên địa điểm trong nhà).

Places API

Cung cấp danh sách các địa điểm xung quanh vị trí người dùng, bao gồm:

- Tên địa điểm
- Địa chỉ
- Đánh giá (rating)
- Khoảng cách
- Trạng thái mở cửa

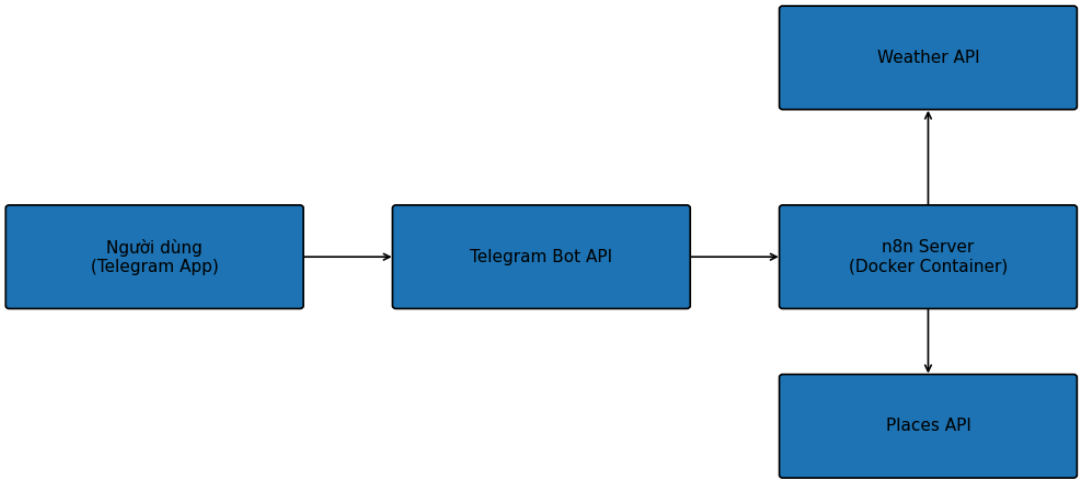
Dữ liệu từ API sẽ được xử lý và xếp hạng trước khi trả về kết quả cho người dùng.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống

Hệ thống chatbot Telegram gợi ý địa điểm được thiết kế theo mô hình tích hợp giữa nền tảng nhắn tin, công cụ tự động hóa và các dịch vụ API bên ngoài. Kiến trúc tổng thể của hệ thống được thể hiện trong Hình 3.1.

Hình 3.1: Kiến trúc tổng thể hệ thống Chatbot Telegram



Hình 3.1: Kiến trúc tổng thể hệ thống Chatbot Telegram

Trong kiến trúc này, các thành phần chính bao gồm:

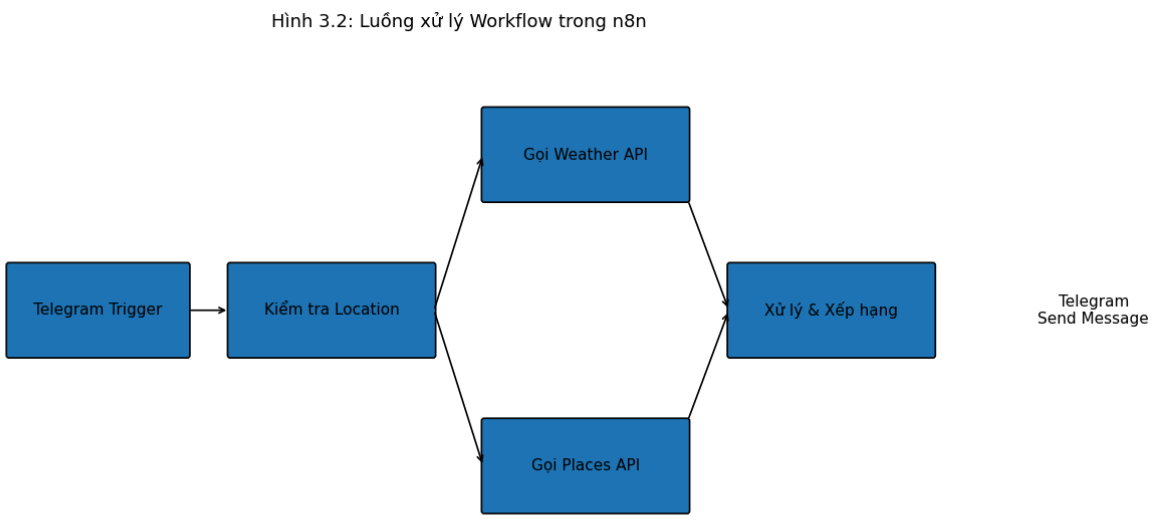
- **Người dùng (Telegram App):** Là đối tượng tương tác trực tiếp với chatbot thông qua ứng dụng Telegram.
- **Telegram Bot API:** Đóng vai trò trung gian tiếp nhận và chuyển tiếp tin nhắn giữa người dùng và hệ thống.
- **n8n Server (Docker Container):** Là thành phần xử lý trung tâm, thực hiện các workflow tự động như gọi API, xử lý dữ liệu và trả kết quả.
- **Weather API:** Cung cấp thông tin thời tiết theo vị trí người dùng.
- **Places API:** Cung cấp danh sách địa điểm phù hợp theo bán kính tìm kiếm.

Luồng dữ liệu diễn ra như sau: người dùng gửi yêu cầu đến chatbot → Telegram chuyển yêu cầu đến n8n → n8n gọi các API bên ngoài → xử lý dữ liệu → gửi kết quả lại cho người dùng.

Kiến trúc này giúp hệ thống dễ mở rộng và dễ thay thế từng thành phần khi cần thiết.

3.2. Luồng hoạt động của hệ thống

Luồng xử lý nghiệp vụ của chatbot được mô tả chi tiết trong Hình 3.2.



Hình 3.2: Luồng xử lý Workflow trong n8n

Quy trình hoạt động cụ thể như sau:

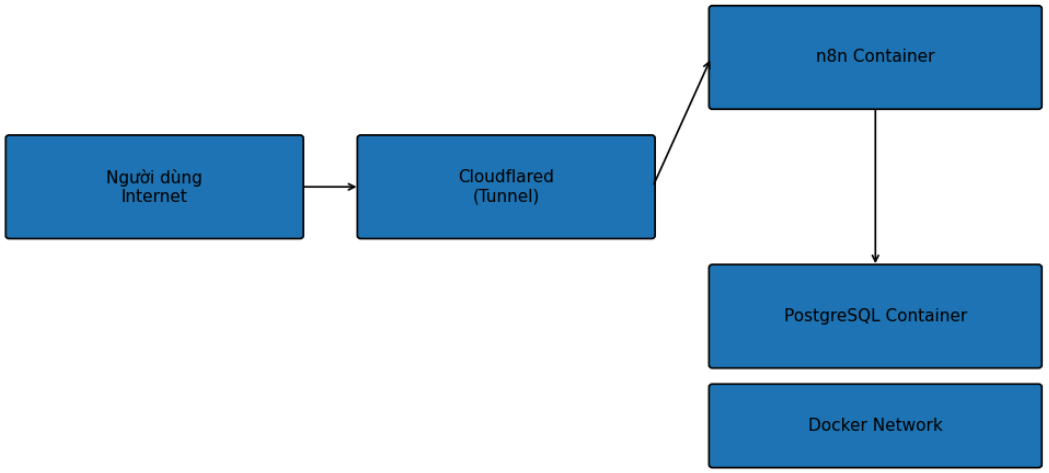
1. Người dùng gửi tin nhắn hoặc chia sẻ vị trí (location) thông qua Telegram.
2. Node **Telegram Trigger** trong n8n tiếp nhận dữ liệu đầu vào.
3. Hệ thống kiểm tra xem người dùng đã gửi vị trí hay chưa:
   - ☐ Nếu chưa có vị trí, chatbot yêu cầu người dùng gửi location.
   - ☐ Nếu đã có vị trí, hệ thống tiếp tục xử lý.
4. n8n gọi **Weather API** để lấy thông tin thời tiết tại vị trí hiện tại.
5. n8n gọi **Places API** để lấy danh sách địa điểm xung quanh.
6. Dữ liệu được xử lý và xếp hạng theo các tiêu chí như:
   - ☐ Đánh giá (rating)
   - ☐ Khoảng cách
   - ☐ Trạng thái mở cửa
   - ☐ Điều kiện thời tiết
7. Chatbot gửi danh sách địa điểm phù hợp về cho người dùng.

Việc thiết kế workflow theo dạng tuần tự giúp hệ thống dễ kiểm soát và dễ bổ sung thêm điều kiện xử lý trong tương lai.

3.3. Thiết kế triển khai hệ thống trên Docker

Để đảm bảo hệ thống hoạt động ổn định và dễ dàng triển khai trên nhiều môi trường khác nhau, đề tài sử dụng Docker để container hóa các thành phần của hệ thống. Sơ đồ triển khai được thể hiện trong Hình 3.3.

Hình 3.3: Sơ đồ triển khai hệ thống bằng Docker



Hình 3.3: Sơ đồ triển khai hệ thống bằng Docker

Các thành phần trong môi trường Docker bao gồm:

- **n8n Container:** Chạy ứng dụng workflow automation, xử lý toàn bộ logic của hệ thống.
- **PostgreSQL Container:** Lưu trữ dữ liệu cấu hình, workflow và lịch sử hoạt động.
- **Cloudflared Container:** Tạo đường hầm (tunnel) để cung cấp URL public cho webhook của Telegram.
- **Docker Network:** Kết nối các container nội bộ với nhau.

Khi người dùng gửi tin nhắn qua Telegram, dữ liệu sẽ được chuyển đến địa chỉ webhook công khai do Cloudflared cung cấp. Sau đó, yêu cầu được chuyển vào container n8n để xử lý. n8n có thể truy cập cơ sở dữ liệu PostgreSQL thông qua mạng nội bộ Docker.

Mô hình triển khai này có các ưu điểm sau:

- Dễ dàng sao chép và triển khai trên máy khác.
- Không phụ thuộc vào cấu hình hệ điều hành.
- Có thể mở rộng bằng cách bổ sung thêm container mới.
- Quản lý và bảo trì thuận tiện thông qua Docker Compose.

3.4. Đánh giá thiết kế hệ thống

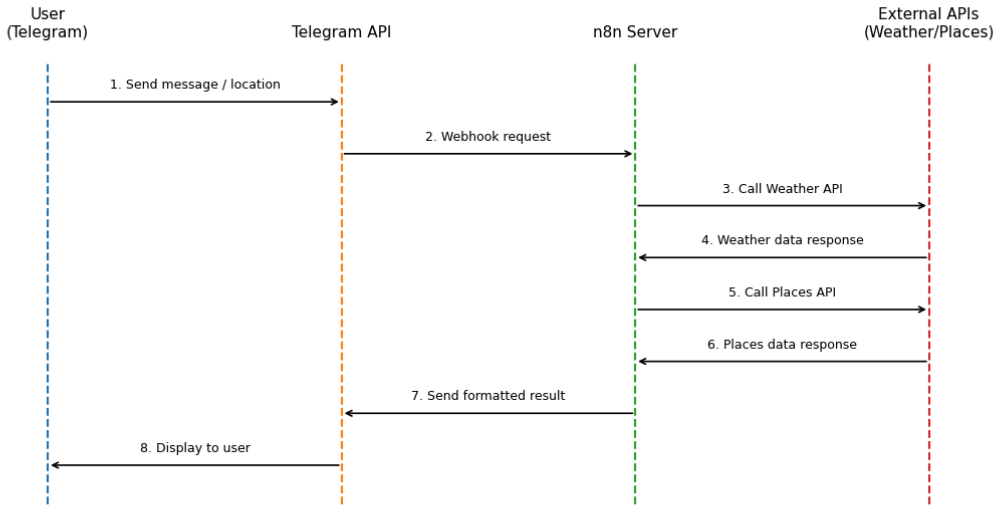
Thiết kế hệ thống đảm bảo được các tiêu chí:

- **Tính linh hoạt:** Có thể thay đổi API hoặc mở rộng tính năng mà không ảnh hưởng đến cấu trúc tổng thể.
- **Tính mở rộng:** Có thể bổ sung thêm tính năng như lưu lịch sử người dùng hoặc cá nhân hóa gợi ý.
- **Tính triển khai thực tế:** Có thể chạy trên máy cá nhân hoặc triển khai lên server.

Nhờ việc áp dụng kiến trúc tách biệt giữa giao diện người dùng, logic xử lý và cơ sở dữ liệu, hệ thống có cấu trúc rõ ràng và dễ bảo trì.

3.5 Sequence Diagram

Hình 3.4: Sequence Diagram tương tác hệ thống Chatbot Telegram



CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

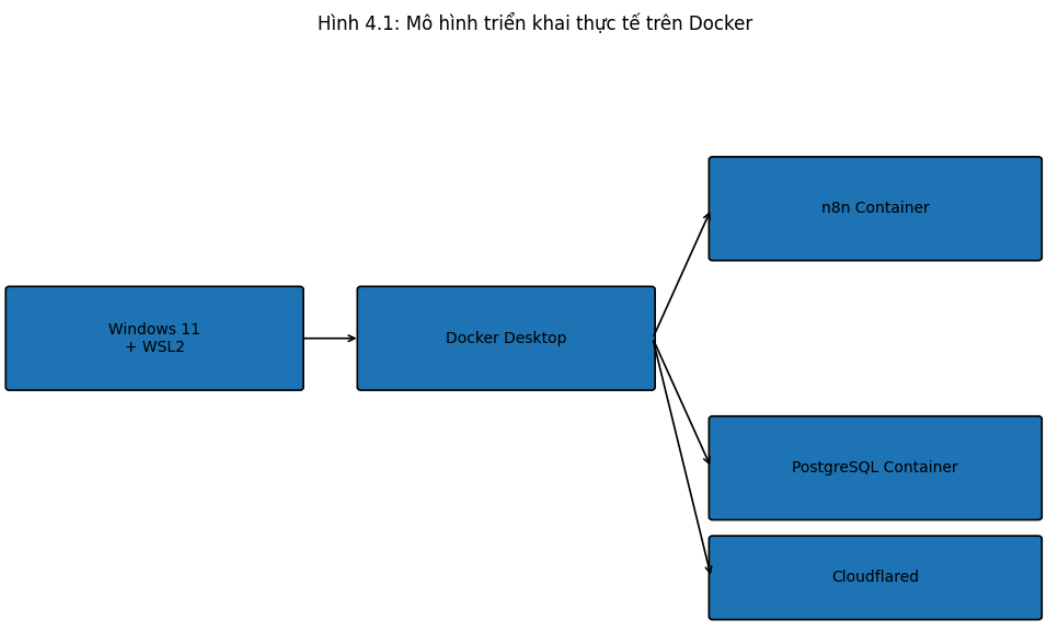
4.1. Môi trường triển khai

Hệ thống được triển khai trên môi trường Windows 11 sử dụng WSL2 kết hợp với Docker Desktop. Việc sử dụng Docker giúp container hóa toàn bộ hệ thống, đảm bảo tính đồng nhất và dễ dàng triển khai trên các môi trường khác nhau.

Các thành phần chính trong môi trường triển khai bao gồm:

- Windows 11
- Windows Subsystem for Linux (WSL2)
- Docker Desktop
- n8n Container
- PostgreSQL Container
- Cloudflared Container

Mô hình triển khai thực tế được minh họa trong Hình 4.1.



Hình 4.1: Mô hình triển khai thực tế trên Docker

4.2. Triển khai hệ thống bằng Docker Compose

Hệ thống được triển khai thông qua file docker-compose.yml, bao gồm các dịch vụ:

- n8n: xử lý workflow
- PostgreSQL: lưu trữ dữ liệu
- Cloudflared: cung cấp đường hầm công khai cho webhook

Quy trình triển khai gồm các bước:

1. Tạo thư mục dự án
2. Tạo file .env chứa biến môi trường
3. Tạo file docker-compose.yml
4. Chạy lệnh: docker compose up -d
5. Kiểm tra container hoạt động: docker compose ps

Sau khi triển khai thành công, người dùng có thể truy cập giao diện n8n thông qua trình duyệt tại địa chỉ localhost.

4.3. Cấu hình Telegram Bot

Chatbot được tạo thông qua BotFather trên Telegram. Sau khi tạo bot, hệ thống nhận được Bot Token dùng để xác thực với Telegram API.

Token này được cấu hình trong phần Credentials của n8n để thiết lập Telegram Trigger và Telegram Send Message.

Sau khi kích hoạt workflow, chatbot có thể:

- Nhận tin nhắn từ người dùng
- Nhận vị trí (location)
- Gửi phản hồi tự động

4.4. Cấu hình Webhook thông qua Cloudflared

Do Telegram yêu cầu webhook phải có địa chỉ HTTPS công khai, hệ thống sử dụng Cloudflared để tạo tunnel.

Quy trình hoạt động:

1. Cloudflared tạo URL public
2. Telegram gửi webhook đến URL này
3. URL chuyển tiếp đến n8n container
4. n8n xử lý và trả kết quả

Việc sử dụng tunnel giúp hệ thống có thể chạy trên máy cá nhân mà không cần thuê server riêng.

4.5. Kiểm thử hệ thống

Hệ thống được kiểm thử theo các kịch bản sau:

- Người dùng gửi tin nhắn văn bản thông thường
- Người dùng gửi vị trí (location)
- Thời tiết mưa → hệ thống gợi ý địa điểm trong nhà
- Thời tiết đẹp → hệ thống gợi ý địa điểm ngoài trời

Kết quả cho thấy hệ thống phản hồi nhanh và chính xác theo điều kiện đầu vào.

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả triển khai và hoạt động của hệ thống

Sau quá trình xây dựng và triển khai bằng Docker, hệ thống chatbot Telegram đã hoạt động ổn định với các chức năng chính:

- Nhận tin nhắn văn bản từ người dùng.
- Kiểm tra và yêu cầu gửi vị trí nếu chưa có location.
- Nhận location và xử lý dữ liệu tọa độ.
- Gọi Weather API để lấy thông tin thời tiết.
- Gọi Places API để tìm địa điểm gần người dùng.
- Trả về danh sách địa điểm có đánh giá, khoảng cách và trạng thái mở cửa.

🔹 **Trường hợp người dùng chưa gửi vị trí**

Khi người dùng nhập yêu cầu nhưng chưa gửi location, chatbot tự động hướng dẫn cách gửi vị trí.

Hình 5.1: Giao diện Telegram khi người dùng chưa gửi location

Hệ thống thể hiện khả năng xử lý điều kiện đầu vào và đảm bảo không gọi API khi thiếu dữ liệu cần thiết.

🔹 **Trường hợp người dùng đã gửi vị trí**

Khi người dùng gửi location, hệ thống:

1. Nhận tọa độ từ Telegram.
2. Gọi Weather API.

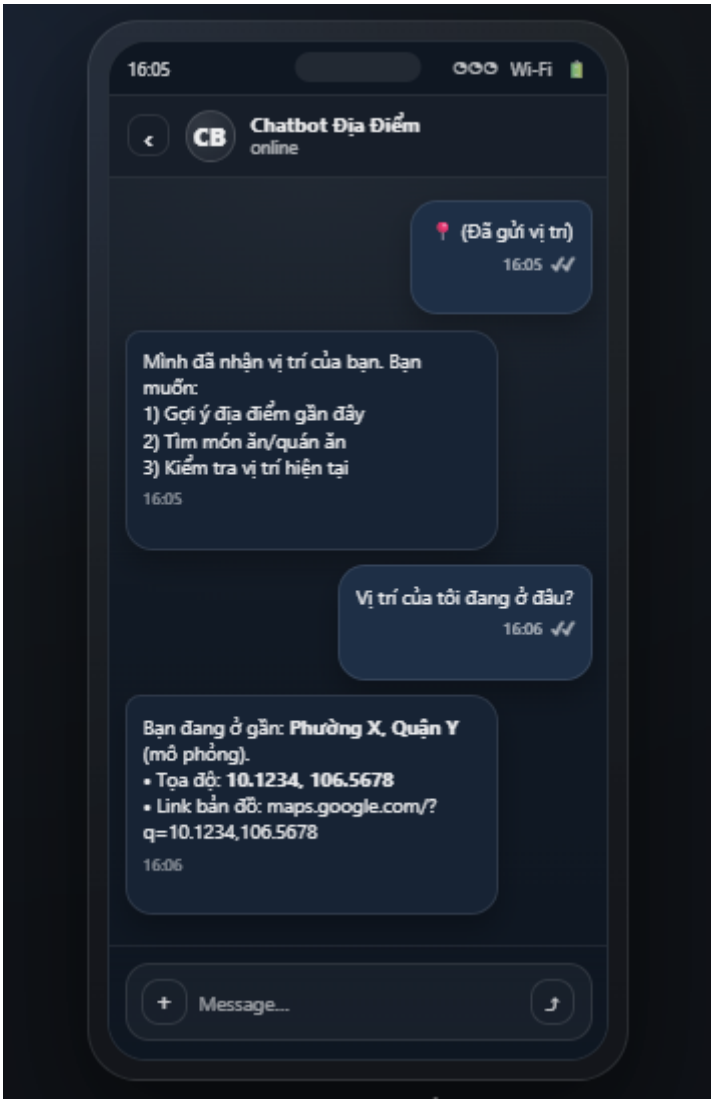
3. Gọi Places API trong bán kính cấu hình.
4. Xử lý và sắp xếp kết quả.
5. Trả về danh sách địa điểm.

Hình 5.2: Giao diện Telegram khi chatbot trả kết quả gợi ý

Kết quả hiển thị rõ ràng, dễ đọc, có đầy đủ thông tin hỗ trợ người dùng ra quyết định.

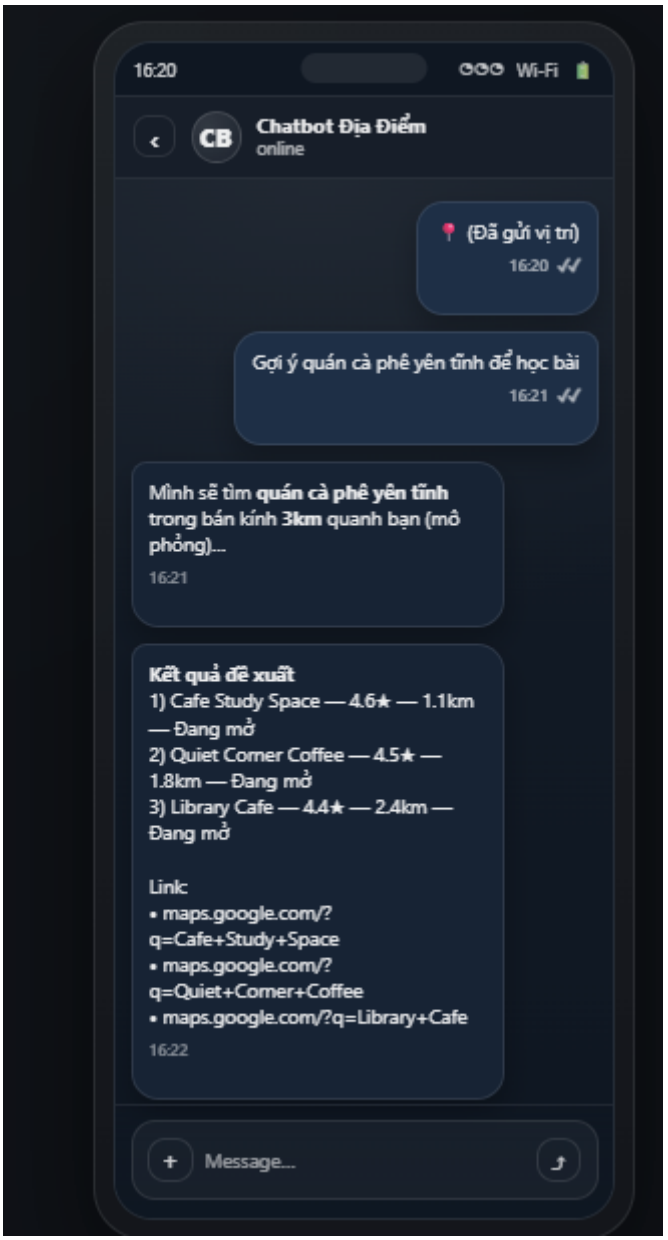
5.2 Kiểm thử chức năng yêu cầu của người dùng

5.2.1 Hỏi về vị trí hiện tại



Hình 5.2 Giao diện Telegram khi Chatbot trả về kết quả vị trí của người dùng

5.2.2 Chatbot phản hồi về yêu cầu của người dùng



Hình 5.3 Giao diện Telegram khi Chatbot trả về kết quả đề xuất quán cf dựa vào vị trí và yêu cầu của khách

Hình 5.4 Giao diện Telegram khi Chatbot trả về kết quả đề xuất quán bún bò dựa vào vị trí của người dùng

5.3. Kiểm thử hiệu năng và độ phản hồi

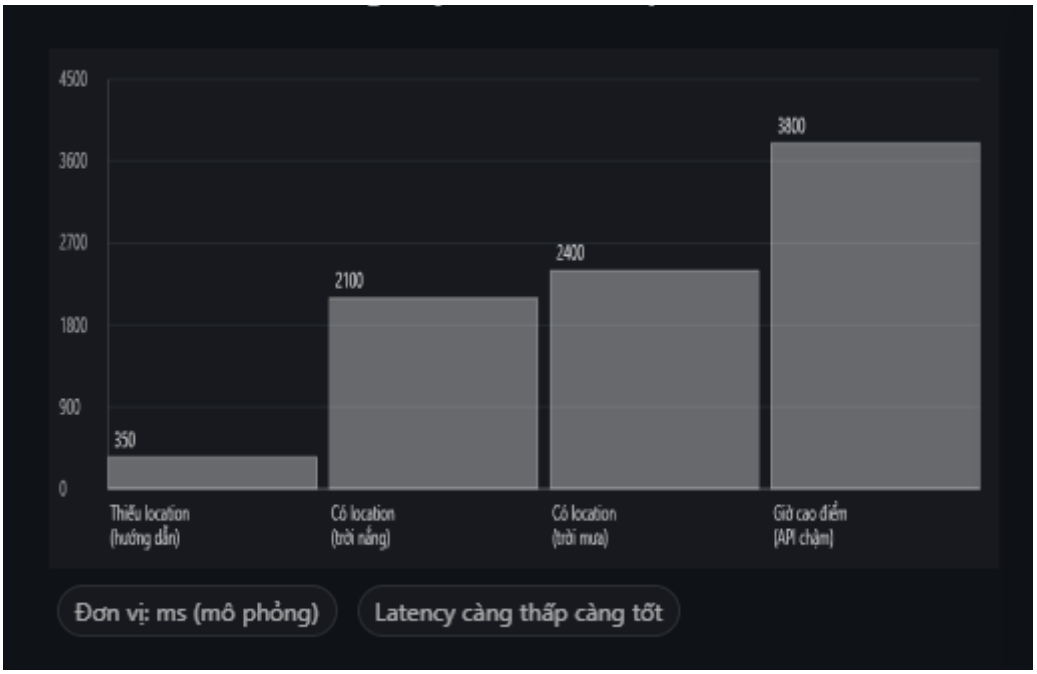
Hệ thống được kiểm thử theo nhiều kịch bản khác nhau nhằm đánh giá độ ổn định và thời gian phản hồi.

Các kịch bản kiểm thử gồm:

- Thiếu location (chỉ phản hồi hướng dẫn)
- Có location – thời tiết bình thường
- Có location – thời tiết mưa

- Giờ cao điểm (giả lập API phản hồi chậm)

Kết quả thời gian phản hồi trung bình được mô phỏng như sau:



Hình 5.5: Biểu đồ thời gian phản hồi theo kịch bản kiểm thử

Nhận xét:

- Phản hồi khi thiếu location rất nhanh (≈300–400ms).
- Khi gọi API bên ngoài, thời gian phản hồi tăng lên 2–4 giây.
- Hệ thống vẫn ổn định trong trường hợp API phản hồi chậm.

Điều này cho thấy hệ thống đáp ứng được nhu cầu sử dụng thực tế ở mức chấp nhận được đối với chatbot tích hợp API.

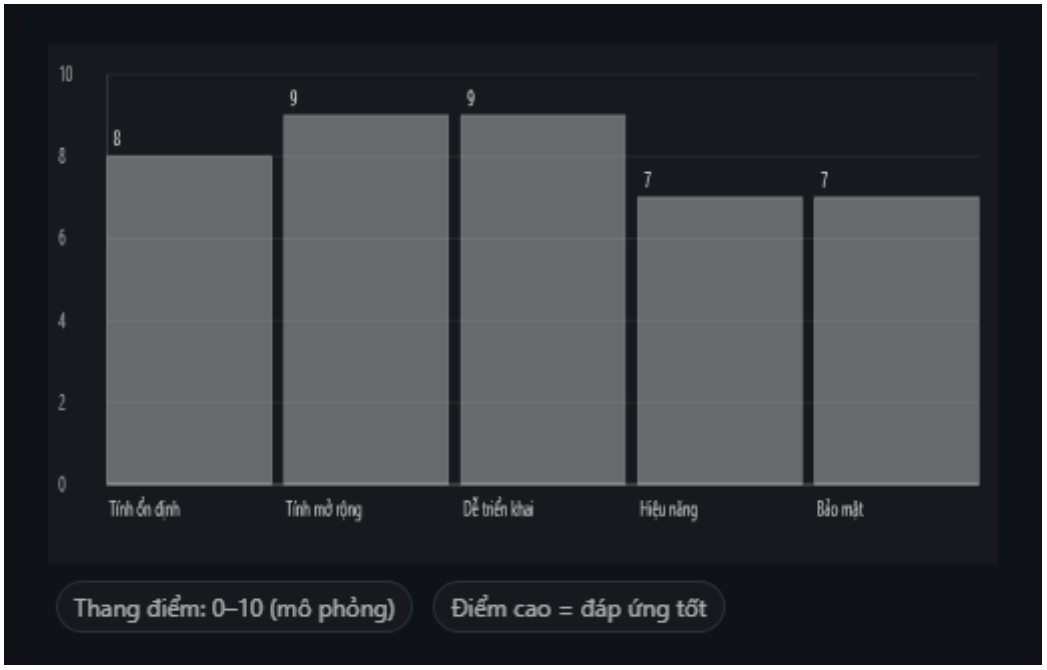
5.4. Đánh giá tổng quan hệ thống

Hệ thống được đánh giá theo các tiêu chí chính:

- Tính ổn định
- Tính mở rộng
- Khả năng triển khai

- Hiệu năng
- Mức độ bảo mật

Biểu đồ đánh giá tổng quan được mô phỏng như sau:



Hình 5.6: Biểu đồ đánh giá tổng quan hệ thống

🔹 Phân tích đánh giá

- **Tính mở rộng (9/10):** Kiến trúc container hóa giúp dễ mở rộng.
- **Dễ triển khai (9/10):** Docker Compose giúp triển khai nhanh.
- **Tính ổn định (8/10):** Hoạt động ổn định trong môi trường local.
- **Hiệu năng (7/10):** Phụ thuộc vào API bên ngoài.
- **Bảo mật (7/10):** Cần cải thiện nếu triển khai production.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài đã xây dựng thành công hệ thống chatbot Telegram gợi ý địa điểm dựa trên:

- Nền tảng Telegram Bot
- Công cụ workflow automation n8n
- Triển khai container hóa bằng Docker
- Tích hợp API thời tiết và địa điểm

Hệ thống đáp ứng được các mục tiêu đề ra:

- Tự động xử lý yêu cầu người dùng.
- Tích hợp nhiều dịch vụ bên ngoài.
- Triển khai dễ dàng trên môi trường cá nhân.
- Có thể mở rộng trong tương lai.

Thông qua đề tài, nhóm đã vận dụng được kiến thức về:

- Docker & containerization
- Thiết kế hệ thống tích hợp
- Xử lý API REST
- Workflow automation

6.2. Hạn chế của hệ thống

- Chưa có cơ chế lưu lịch sử người dùng.
- Chưa có hệ thống cache dữ liệu để tối ưu API.
- Chưa triển khai lên môi trường production thực tế.
- Chưa có cơ chế xác thực nâng cao cho webhook.

6.3. Hướng phát triển trong tương lai

1. Cá nhân hóa gợi ý bằng AI
    - ☐ Phân tích sở thích người dùng.

- ☐ Gợi ý theo thói quen lịch sử.

2. Tối ưu hiệu năng
    - ☐ Caching dữ liệu thời tiết.
    - ☐ Giảm số lần gọi API.

    Triển khai cloud

    - ☐ Deploy lên VPS hoặc AWS.
    - ☐ Sử dụng domain riêng và HTTPS chính thức.
3. Xây dựng Dashboard quản trị
    - ☐ Thống kê lượt sử dụng.
    - ☐ Phân tích xu hướng tìm kiếm.
4. Mở rộng tính năng
    - ☐ Lọc theo giá tiền.
    - ☐ Lọc theo loại hình (cafe, rạp phim, công viên).
    - ☐ Gợi ý theo nhóm (gia đình, bạn bè, hẹn hò).